# Rung 1 — From a sine on the scope to *how wrong is it, provably*

**Where you start:** you can compute a sine, you know binary and fixed-point numbers, and you've seen a signal on a scope. That's everything you need for this rung.

**What you'll be able to do by the end:** look at a real hardware sine, say *exactly* how much it can differ from the true sine because of the arithmetic — and know that number is a **proven ceiling**, not a guess from testing.

---

## 1. The bench

We put a real transcendental unit — `eml_sin`, the sine hardware from the Forge compiler — on a real Arty A7 FPGA. It sweeps `x` from 0 to ~1 and, for each `x`, computes `sin(x)` in hardware. We captured **1201 samples** over the wire. (That's the `sin_certificate_v0` kernel; its evidence packet is in the repo.)

Here's one captured sample:

```
x = 0.406250      sin_hw(x) = 0.395142      true sin(x) = 0.395164
```

The hardware is **close** — but not exact. Off by about `0.000022` here. Two honest questions:

1. *How wrong can it get?*
2. Can we put a **proven** number on that — a ceiling no input can ever exceed — instead of "we tested a lot of values and it looked fine"?

That second question is the whole game. Testing shows you the cases you tried. A proof covers the cases you *didn't*.

## 2. The one new idea: fixed-point numbers carry a tiny, bounded error

The hardware doesn't use real numbers. It uses **Q16.16 fixed-point**: a 32-bit integer where the real value is `integer / 2¹⁶`.

- `2¹⁶ = 65536`.
- The smallest step it can represent — **1 LSB** (least-significant bit) — is `1/65536 ≈ 0.0000153`.

So Q16.16 lays a grid over the number line with spacing `1/65536`. Any real value that isn't exactly on the grid gets **rounded down to the nearest grid line** (truncated). The most you can lose doing that is one grid step:

> **Truncation error of one operation ≤ 1 LSB = 1/65536.**

That's the entire idea of Rung 1. Everything below is just applying it.

### Where truncation happens: `qmul`

Multiplying two Q16.16 numbers is the interesting case. If `a` and `b` are each `value × 2¹⁶`, then `a × b` is `value × 2³²` — it has **32** fractional bits, not 16. To get back to Q16.16 you shift right by 16, i.e. **throw away the bottom 16 bits**:

```
qmul(a, b) = (a * b) >>> 16      // drop the low 16 bits
```

Those dropped 16 bits are a fraction less than 1 LSB. So **every `qmul` is off from the exact product by less than 1 LSB.** That's it. That's the atom.

## 3. Do it by hand

`eml_sin` computes its answer with a short chain of `qmul`s (squaring `x`, building `x³` and `x⁵`, scaling by `1/6` and `1/120`). Call it a handful of truncations.

Naively you'd think: *a handful of qmuls, each ≤ 1 LSB, so the total ≤ a-handful of LSBs.* That's the right instinct. The catch is that an early truncation gets **carried along and multiplied** by later operations, so the errors don't just add — they *compound* through the chain. Working out that worst-case compounding is exactly what a proof is for (that's Rung 3).

When you do work it out, the fixed-point part of `eml_sin`'s error is pinned at **18 LSBs**:

```
fixed-point error  ≤  18 / 2¹⁶  =  18 / 65536  ≈  0.000275
```

Hold onto that number: **`18/D`**, with `D = 2¹⁶`. It's the first half of the certificate.

## 4. Make it formal

In machlib (the Lean proof library), that ceiling isn't asserted — it's **derived**. The `>>> 16` truncation is modeled with the `floor` function, and a single lemma says "flooring loses less than 1 LSB." The proof then walks the `qmul` chain, composing those per-step losses into the `18/D` total. The statement you end up with is one piece of:

```
MachLib.Real.eml_sin_full_fwd_error :
    |eml_sin_rtl(x, D) - sin(x)|  ≤  18/D + x⁶      for x in [0,1],  D = 2¹⁶
                                    ^^^^
                                    the fixed-point part you just derived
```

"Machine-checked" means a computer verified every step of that derivation with no gaps — that's what **`sorryAx`-free** means (no `sorry`, no unproven axioms sneaking in). You'll write your own tiny version of a proof like this on Rung 4.

## 5. Watch it hold

Back to the bench. That `18/D ≈ 0.000275` is the **ceiling** from arithmetic alone. What did the real silicon actually do?

```
Across 1201 hardware samples:  worst observed |error| ≈ 0.000200
Tightest margin (ceiling − observed) ≈ 0.000258, at small x
```

The observed error stayed **under the proven ceiling on every one of 1201 samples**. The certificate held on real hardware. That's the payoff: a number you can *trust for inputs you never tested*, confirmed by a scope trace.

## The cliffhanger → Rung 2

You may have noticed the certificate had **two** terms: `18/D + x⁶`. You just derived the `18/D` — the fixed-point part. But where does the `x⁶` come from?

It's a *different* source of error. The hardware doesn't compute the "real" sine at all — no circuit can. It uses an **approximate formula**: the first three terms of the Taylor series, `x − x³/6 + x⁵/120`. That approximation is wrong by an amount that grows like `x⁶` — which is why our observed error crept up as `x` got bigger.

To *bound* that approximation error — to know the `x⁶` is a real ceiling and not a guess — you need one genuinely new piece of math: **Taylor's theorem with a remainder.** That's the first real-analysis idea, and it needs one skill first: reading a proof.

→ **Next:** Rung 1.5 — *What even is a proof?* — then Rung 2 — *Where the `x⁶` comes from.*

## Exercises & checkpoint

1. Compute **1 LSB** in Q16.16 as a decimal. Then compute the fixed-point ceiling `18/D` (`D = 2¹⁶`). About how many LSBs is that?
2. You multiply two Q16.16 numbers and shift right by 16. In the worst case, how far is the result from the exact product — and *why* is it less than 1 LSB, not more?
3. Our capture's worst observed error was ≈ `2e-4`, and the fixed-point ceiling is ≈ `2.75e-4`. The observed error is *under* the ceiling — but there's also the `x⁶` term (next rung). At `x = 0.1`, is `x⁶` or `18/D` the bigger part of the ceiling? At `x = 0.9`?

**Checkpoint:** you can say, in one sentence, what `18/D` represents, and compute it as a decimal.